

5. 토큰화

- 자연어: 인공적으로 만들어진 프로그래밍 언어가 아닌, 사람들의 언어 활동을 위해 자연히 만들어진 인간의 언어
- 자연어 처리(Natural Language Processing, NLP): 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술

자연어 처리 모델 개발을 위해 해결해야하는 문제

- 모호성: 단어와 구가 사용되는 맥락에 따라 여러 의미를 지니는 모호성을 해결할 수 있어야 함
- 가변성: 인간의 언어의 특징인 가변성을 처리할 수 있어야 함
- 구조: 문장의 구조와 문법적 요소를 이해하여 의미를 추론하거나 분석

자연어 처리 모델을 만들기 위해서는 말뭉치를 일정한 단위인 토큰으로 나누어야 함.

- 말뭉치(Corpus):
 - 자연어 모델을 훈련하고 평가하는 데 사용되는 대규모의 자연어
 - 목적에 따라 구축되는 텍스트 데이터
- 토큰: 개별 단어나 문장 부호와 같은 텍스트 (말뭉치보다 더 작은 단위)
- 토큰화(Tokenization): 컴퓨터가 자연어를 이해할 수 있도록 토큰으로 나누는 과정
- 토큰나이저(Tokenizer): 텍스트 문자열을 토큰으로 나누는 알고리즘 or 소프트웨어

토큰나이저를 구축하는 방법

- 공백 분할 : 공백 단위로 분리
- 정규표현식 적용
- 어휘 사전 적용
 - 사전에 정의된 단어를 활용해 토큰나이저를 구축하는 방법
 - OOV(Out of Vocab): 직접 어휘 사전을 구축하였기 때문에 존재하지 않는 단어나 토큰
 - 큰 어휘 사전을 구축하면 학습 비용의 증대와 차원의 저주 문제에 빠질 수 있다
- 머신러닝 활용

단어 및 글자 토큰화

- 토큰화: 텍스트 데이터를 구조적으로 분해하여 개별 토큰으로 나누는 작업
→ 정확한 분석을 위한 필수 과정이며, 단어나 문장의 빈도, 출현 패턴 등을 파악 가능

단어 토큰화

- 텍스트 데이터를 의미 있는 단위인 단어로 분리하는 작업
- 띄어쓰기, 문장 부호, 대소문자 등의 특정 구분자를 활용해 토큰화 수행
- 활용 범위: 품사 태깅, 개체명 인식, 기계 번역 등
- 가장 일반적인 토큰화 방법
- `split()` 메서드를 활용한다.

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"  
tokenized = review.split()  
print(tokenized)
```

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"  
tokenized = review.split()  
print(tokenized)  
... ['현실과', '구분', '불가능한', 'cg.', '시각적', '즐거움은', '최고!', '더불어', 'ost는', '더더욱', '최고!!']
```

- 단점: 한국어 접사, 문장 부호, 오타 혹은 띄어쓰기 오류 등에 취약함

글자 토큰화 (Character Tokenization)

- 띄어쓰기뿐만 아니라 글자 단위로 문장을 나누는 방식
- 장점
 - 비교적 작은 단어 사전 구축 가능 → 컴퓨터 자원 절약
 - 전체 말뭉치 학습 시 각 단어 학습 빈도 증가
- 활용 범위: 언어 모델링과 같은 시퀀스 예측 작업 (ex. 다음 문자를 예측하는 언어 모델링)
- `list()` 메서드를 활용한다.

```
tokenized2 = list(review)
print(tokenized2)
```

```
['현', '실', '과', ' ', '구', '분', ' ', '불', '가', '능', '한', ' ', 'c', 'g', ' ', ' ', ' ', '시', '각', '적', ' '
```

- 공백도 토큰으로 나뉨.
- 영어의 경우 각 알파벳으로 나뉘고, 한글의 경우 하나의 글자 단위로 나뉨
- 한글의 경우 하나의 글자는 여러 자음과 모음의 조합으로 이루어져 있기 때문에 자소 단위 토큰화를 진행하기 위해서 자모 라이브러리를 활용
- 자모 라이브러리: 한글 문자 및 자모 작업을 위한 한글 음절 분해 및 합성 라이브러리

```
# 자모 변환 함수
retval = jamo.h2j(
    hangul_string
)
```

컴퓨터가 한글을 인코딩하는 두 가지 방식

- 조합형: 글자를 자모 단위로 나눠 인코딩한 뒤 한글을 표현
- 완성형: 조합된 글자 자체에 값을 부여해 인코딩하는 방식
- **h2j** 함수: 완성형으로 입력된 한글을 조합형 한글로 변환한다.
- **j2hcj** 함수: 조합형 한글 문자열을 자소 단위로 나눠 반환하는 함수
 - 조합형 한글로 입력된 문자열은 초성, 중성, 종성으로 나뉜다.
 - 이 함수를 통해 완성형 한글을 쉽게 자소 단위로 나눌 수 있다.

형태소 토큰화

- 텍스트를 형태소 단위로 나누는 토큰화 방법
- 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미 있는 단위로 분류하는 작업
- 띄어쓰기로 구분이 어려운 한국어와 같은 교착어에서 형태소 토큰화는 특히 중요하게 수행됨.

관련 용어들

- 형태소 : 실제로 의미를 가지고 있는 최소의 단위

	의미	종류	예시
자립 형태소	스스로 의미를 가지고 있는 형태소	명사, 동사, 형용사	그, 나, 인사
의존 형태소	자립 형태소와 함께 조합되어 문장에서 특정한 역할을 수행하는 형태소	조사, 어미, 접두사, 접미사	-는, -에게, -를, 했-, -다

형태소 어휘 사전

- 각 단어의 형태소 정보를 포함하는 어휘 사전
- 단어가 어떤 형태소들의 조합으로 이루어져 있는지에 대한 정보를 담고 있어 형태소 분석 작업에서 매우 중요한 역할을 한다
- 품사 태깅: 텍스트 데이터를 형태소 분석하여 각 형태소에 해당하는 품사를 태깅하는 작업
 - 자연어 처리 분야에서 문맥 고려를 가능하게 함

KoNLPy

- 한국어 자연어 처리를 위해 개발된 라이브러리
- 기능: 명사 추출, 형태소 분석, 품사 태깅
- 지원하는 형태소 분석기: Okt, 꼬꼬마, 코모란, 한나눔, 메캅 등

```

from konlpy.tag import Okt
okt = Okt()
sentence = "무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."
nouns = okt.nouns(sentence)
phrases = okt.phrases(sentence)
morphs = okt.morphs(sentence)
pos = okt.pos(sentence)

print("명사 추출 :", nouns)
print("구 추출 :", phrases)
print("형태소 추출 :", morphs)
print("품사 태깅 :", pos)

```

- 형태소 분석과 품사 태깅은 주요 키워드를 추출하거나 문장의 긍정/부정 여부를 판단하는 언어 모델의 성능을 높이는 데도 중요한 역할을 한다.

NLTK

- 자연어 처리를 위해 개발된 라이브러리
 - 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석
- NLTK 주로 영어 자연어 처리를 위해 개발됐지만, 네덜란드어, 프랑스어, 독일어 등과 같은 다양한 언어의 자연어 처리를 위한 데이터와 모델을 제공한다.
- 트리뱅크(Treebank): 대규모의 영어 말뭉치

spaCy

- 사이썬 기반으로 개발된 오픈 소스 라이브러리
- 자연어 처리를 위한 기능 제공
- 빠른 속도와 높은 정확도를 목표로 하는 머신러닝 기반의 자연어 처리 라이브러리

```
import spacy
nlp = spacy.load("en_core_web_sm")
sentence = "Those who can imagine anything, can create the impossible."
doc = nlp(sentence)
for token in doc:
    print(f"[{token.pos_:5}-{token.tag_:3}] : {token.text}")
```

```
[PRON -DT ] : Those
[PRON -WP ] : who
[AUX -MD ] : can
[VERB -VB ] : imagine
[PRON -NN ] : anything
[PUNCT- , ] : ,
[AUX -MD ] : can
[VERB -VB ] : create
[DET -DT ] : the
[ADJ -JJ ] : impossible
[PUNCT-. ] : .
```

하위 단어 토큰화

발생 배경

- 형태소는 자연어의 최소 의미 단위이며, 대부분의 자연어가 형태소의 조합으로 이뤄져 있다.

- 형태소 분석기는 전문용어나 고유어가 많은 데이터를 처리할 때 약점을 보인다.
→ 모르는 단어를 적절한 단위로 나누는 것에 취약하며, 이는 잠재적으로 어휘 사전의 크기를 크게 만들고 OOV에 대응하기 어렵게 만든다!
- 하지만 현대 자연어 처리에서 신조어, 오타자, 축약어 등을 고려해야 하기 때문에 이와 같은 문제 해결 방법 중 하나인 것이 **하위 단어 토큰화(Subword Tokenization)**

하위 단어 토큰화란?

- 하나의 단어가 빈번하게 사용되는 하위 단어의 조합으로 나누어 토큰화하는 방법
 - ex) 'Reinforcement' → 'Rein', 'force', 'ment'
- 하위 단어 토큰화를 적용하면 단어의 길이를 줄일 수 있어 처리 속도를 향상한다.

바이트 페어 인코딩(BPE, Byte Pair Encoding)

- 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 조합 탐지와 부호화를 반복 → 이 과정에서 자주 등장하는 단어는 하나의 토큰으로 토큰화, 덜 등장하는 단어는 여러 토큰의 조합으로 표현됨!
- 입력 데이터에서 가장 많이 등장한 글자의 빈도수를 측정하고, 가장 빈도수가 높은 글자 쌍을 탐색하여 치환하는 과정을 반복함



토큰라이저로서 바이트 페어 인코딩

- 자주 등장하는 글자 쌍을 찾아 치환하는 대신 **어휘 사전에 추가한다.**

- **센텐스피스**
 - 구글에서 개발한 오픈소스 하위 단어 토큰라이저 라이브러리
 - 바이트 페어 인코딩과 유사한 알고리즘을 사용하여 입력 데이터를 토큰화하고 단어 사전을 생성

토큰라이저 모델 학습

- 센텐스피스 라이브러리를 통해 바이트 페어 인코딩을 수행하는 토큰라이저 모델 학습

1. 청와대 청원 데이터 다운로드

```

from Korpora import Korpora
corpus = Korpora.load("korean_petitions")
dataset = corpus.train
petition = dataset[0]
print("청원 시작일: ", petition.begin)
print("청원 종료일: ", petition.end)
print("청원 동의 수: ", petition.num_agree)
print("청원 범주: ", petition.category)
print("청원 제목: ", petition.title)
print("청원 본문: ", petition.text[:30])

```

```

청원 시작일: 2017-08-25
청원 종료일: 2017-09-24
청원 동의 수: 88
청원 범주: 육아/교육
청원 제목: 학교는 인력센터, 취업센터가 아닙니다. 정말 간곡히 부탁드립니다.
청원 본문: 안녕하세요. 현재 사대, 교대 등 교원양성학교들의 예비

```

- `Korpora.load` : 말뭉치 데이터를 다운로드
- `corpus` 는 총 433,631개의 청원 데이터가 저장되어 있으며, `train` 속성으로 학습 데이터를 가져올 수 있다.

2. 학습 데이터셋 생성

```

# 학습 데이터셋 생성
petitions = corpus.get_all_texts()
with open("/content/drive/MyDrive/corpus.txt", "w", encoding="utf-8") as f:
    for petition in petitions:
        f.write(petition+"\n")

```

```

from sentencepiece import SentencePieceProcessor
tokenizer = SentencePieceProcessor()
tokenizer.load("petition_bpe.model")
sentence = "안녕하세요, 토큰라이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토큰라이저를 사용할 수 있습니다"]
tokenized_sentence = tokenizer.encode_as_pieces(sentence)
tokenized_sentences = tokenizer.encode_as_pieces(sentences)

```

- `get_all_texts` : 본문 데이터셋을 한 번에 불러오기

3. 토큰라이저 모델 학습

```
from sentencepiece import SentencePieceTrainer

SentencePieceTrainer.Train(
    "--input=/content/drive/MyDrive/corpus.txt#
    --model_prefix=petition_bpe#
    --vocab_size=8000 model_type=bpe"
)
```

- `SentencePieceTrainer` : 센텐스피스 라이브러리, 토큰나이저 모델 학습을 진행
- `Train` 메서드로 토큰나이저 모델 학습
- 센텐스피스는 문자열 입력을 통해 인자들을 전달 받음

4. 토큰화 수행

```
from sentencepiece import SentencePieceProcessor
tokenizer = SentencePieceProcessor()
tokenizer.load("petition_bpe.model")
sentence = "안녕하세요, 토큰라이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토큰라이저를 사용할 수 있습니다"]
tokenized_sentence = tokenizer.encode_as_pieces(sentence)
tokenized_sentences = tokenizer.encode_as_pieces(sentences)
```

```
print("단일 문장 토큰화: ", tokenized_sentence)
print("여러 문장 토큰화: ", tokenized_sentences)
```

... 단일 문장 토큰화: ['__안녕하세요', ',', '__토', '크', '나', '이', '저', '가', '__잘', '__학', '']
 여러 문장 토큰화: [['__이렇게', '__입', '력', '값을', '__리', '스트', '로', '__받아서'], ['__쉽', '']]
 단일 문장 정수 인코딩: [667, 6553, 994, 6880, 6544, 6513, 6590, 6523, 161, 110, 6554, 872, 787, 6554]
 여러 문장 정수 인코딩: [[372, 182, 6677, 4433, 1772, 1613, 6527, 4162], [1681, 994, 6880, 6544, 6554]]
 정수 인코딩에서 문장 변환: 안녕하세요, 토큰라이저가 잘 학습되었군요!
 하위 단어 토큰에서 문장 변환: ['이렇게 입력값을 리스트로 받아서', '쉽게 토큰라이저를 사용할 수']

- `encode_as_pieces` 메서드: 문장을 토큰화
- `encode_as_ids` 메서드: 토큰을 정수(어휘 사전의 토큰에 매핑된 ID값, 이를 이용해 자연어 처리 모델 구축)로 인코딩해 제공

- `decode_ids`, `decode_pieces` 메서드: 토크나이저 모델이나 자연어 처리 모델에서 나온 정수를 문자열 데이터로 변환

5. 어휘 사전 불러오기

```
# 어휘 사전 불러오기
from sentencepiece import SentencePieceProcessor

tokenizer = SentencePieceProcessor()
tokenizer.load("petition_bpe.model")

vocab = {idx: tokenizer.id_to_piece(idx) for idx in range(tokenizer.get_piece_size())}
print(list(vocab.items())[:5])
print("vocab size: ", len(vocab))

[(0, '<unk>'), (1, '<s>'), (2, '</s>'), (3, '니다'), (4, '___이')]
vocab size: 8000
```

- `get_piece_size` 메서드: 셴텐스피스 모델에서 생성된 하위 단어의 개수를 반환
- `id_to_piece` : 메서드 정숫값을 하위 단어로 변환 → 하위 단어의 개수만큼 반복해 하위 단어 딕셔너리를 구성
- `<unk>` : unknown의 약자, OOV 발생 시 매핑되는 토큰
- `<s>`, `</s>` : 문장의 시작과 종료 지점을 표시하는 토큰

워드피스(Wordpiece)

- 바이트 페어 인코딩 토크나이저와 유사하지만 빈도 기반이 아닌 확률 기반으로 글자 쌍을 병합
- 모델이 새로운 하위 단어 생성할 때 이전 하위 단어와 함께 나타날 확률을 계산해 가장 높은 확률을 가진 하위 단어를 선택
-

수식 5.1 글자 쌍 병합 점수 수식

$$score = \frac{f(x,y)}{f(x)f(y)}$$

- f: 빈도를 나타내는 함수
- x, y : 병합하려는 하위 단어
- f(x,y): x와 y가 조합된 글자 쌍의 빈도
- score: x와 y를 병합하는 것이 적절한지를 판단하기 위한 점수

토큰나이저스

- 토큰나이저스 라이브러리의 워드피스 API를 이용하면 쉽고 빠르게 토큰나이저 구현 및 학습 가능
- 정규화(Normalization)와 사전 토큰화(Pre-tokenization)를 제공
- 정규화: 일관된 형식으로 텍스트를 표준화하고 모호한 경우 방지를 위해 일부 문자를 대체하거나 제거
 - 불필요한 공백 제거, 대소문자 변환, 유니코드 정규화, 구두점 처리, 특수 문자 처리 등
- 사전 토큰화: 입력 문장을 토큰화하기 전에 단어와 같은 작은 단위로 나누는 기능
 - 공백 혹은 구두점을 기준으로 입력 문장을 나눠 텍스트 데이터를 효율적으로 처리하고 모델의 성능을 향상

1. 워드피스 토큰나이저 학습

```
from tokenizers import Tokenizer
from tokenizers.models import WordPiece
from tokenizers.normalizers import Sequence, NFD, Lowercase
from tokenizers.pre_tokenizers import Whitespace

tokenizer = Tokenizer(WordPiece())
tokenizer.normalizer = Sequence([NFD(), Lowercase()])
tokenizer.pre_tokenizer = Whitespace()

tokenizer.train(["/content/drive/MyDrive/corpus.txt"])
tokenizer.save("petition_wordpiece.json")
```

- 토큰나이저스 라이브러리의 토큰나이저로 워드피스 모델을 불러오기
- 정규화 방식은 normalizers 모듈에 포함된 클래스를 불러와 Sequence 형식으로 인스턴스 전달
 - `NFD()` : NFD 유니코드 정규화
 - `Lowercase()` : 소문자 변환

- 사전 토큰화 방식도 `pre_tokenizers` 모듈에 포함된 클래스를 불러와 적용
 - 공백과 구두점 기준 분리

2. 워드피스 토큰화

```

from tokenizers import Tokenizer
from tokenizers.decoders import WordPiece as WordPieceDecoder

tokenizer = Tokenizer.from_file("/content/petition_wordpiece.json")
tokenizer.decoder = WordPieceDecoder()

sentence = "안녕하세요, 토큰라이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토큰라이저를 사용할 수 있습니다"]

encoded_sentence = tokenizer.encode(sentence)
encoded_sentences = tokenizer.encode_batch(sentences)

print("인코더 형식: ", type(encoded_sentence))
print("단일 문장 토큰화: ", encoded_sentence.tokens)
print("여러 문장 토큰화: ", [enc.tokens for enc in encoded_sentences])

print("단일 문장 정수 인코딩: ", encoded_sentence.ids)
print("여러 문장 정수 인코딩: ", [enc.ids for enc in encoded_sentences])

print("정수 인코딩에서 문장 변환: ", tokenizer.decode(encoded_sentence.ids))

... 인코더 형식: <class 'tokenizers.Encoding'>
단일 문장 토큰화: ['안녕하세요', ' ', ' ', '토', '##크', '##나이', '##저', '##가', '잘', '학습', '##']
여러 문장 토큰화: [['이렇게', '입력', '##값을', '리스트', '##로', '받아서'], ['쉽게', '토', '##']]
단일 문장 정수 인코딩: [8760, 11, 8693, 8415, 16269, 7536, 7488, 7842, 15016, 8670, 8734, 0]
여러 문장 정수 인코딩: [[8187, 19643, 13834, 28119, 7495, 12607], [9739, 8693, 8415, 16269, 753]]
정수 인코딩에서 문장 변환: 안녕하세요, 토큰라이저가 잘 학습되었군요!

```

- `encode` : 인코딩 메서드로 문장 토큰화
- `encode_batch` : 인코딩 배치 메서드로 여러 문장을 한 번에 토큰화
- `tokens` : 토큰화된 데이터는 토큰 속성을 통해 값 확인
- `ids` : 토큰 정수 속성으로 인코딩된 문장의 ID 값 출력
- `decode` : 디코딩 메서드로 정수 인코딩된 결과를 다시 문장으로 디코딩해 출력

최근 연구 동향은 더 큰 말뭉치를 사용해 모델을 학습하고 OOV의 위험을 줄이기 위해 하위 단어 토큰화를 활용한다.